

Telegram Backed Hybrid Cloud Storage Management System

Problem Statement

Commercial cloud storage is expensive due to high subscription fees and egress tariffs. Conversely, the free Telegram Bot API offers unstructured, high-capacity file hosting but lacks multi-user access control, relational folder structures, and web management portals. This project builds a secure middleware proxy pipeline connecting a web interface to a private Telegram channel, using an SQL metadata matrix to map anonymous cloud file keys into user-specific virtual folder trees

Functional Dependencies

- **Credentials → JWT:** Valid user login dictates the generation of a secure stateless session token.
- **Folder Selection → UI Refresh:** Clicking a folder triggers a database query filtering by Parent_Folder_ID to render current subfolders and files.
- **Upload Action → Memory Buffering:** File selection initiates backend multipart streaming to process binary chunks without server memory crashes.
- **Transfer Success → ID Capture:** Successful bot-to-channel transmission dictates the return of a unique, persistent Telegram file_id.
- **Token Receipt → DB Indexing:** Receiving the file_id dictates the immediate entry of file metadata records into the SQL database.

Scope:

- Web dashboard for user authentication, file uploads, downloads, and search.
- Backend proxy to stream chunks asynchronously without local server file persistence.
- Relational database schema mapping Users → Folders → Files to cloud references.

Constraints:

- **Size Cap:** Telegram Bot API limits standard HTTP transfers to 50MB uploads and 20MB downloads (2GB absolute cap via stream chunking).
- **Rate Limits:** External API throughput bottlenecks trigger HTTP 429 throttling errors.
- **Metadata Risk:** Deleting the local database completely unlinks the remote files, making them unidentifiable.